Session 26 Homework — Issue Fields, Delete, and the Backlog

CORE, STRETCH, and CHALLENGE are all required — every one of them must be completed. The three names describe increasing difficulty only, nothing about which parts you submit.

Context (required)

By the end of class, you have a real, working `IssueEditPanel` that can edit an issue's title and status in place, a real persisted drag-and-drop Board, and a real Sprint lifecycle (create, start, complete). The remaining product flow must connect those pieces: an issue's `priority`, `assignee`, and `storyPoints` fields exist on the backend but nothing lets you edit them, and there is no way to delete an issue, see which issues are NOT in any sprint, assign Backlog work to a Sprint, or show only the active Sprint on the Board. This homework closes those gaps (CORE), then transfers the Board's own visual pattern to one new place (STRETCH), then asks you to reason in writing about the Sprint domain's own rules (CHALLENGE).

Goal (required)

`IssueEditPanel` can edit priority, assignee, and story points, not only title and status (CORE); a real way to delete an issue (CORE); a real Backlog page listing every issue not currently in a sprint, assign it to a Sprint, and show only the active Sprint's issues on the Board (CORE); a real story-points badge visible on the Board itself (STRETCH); a real, independent written answer explaining the Sprint lifecycle and its two-layer one-active-Sprint guarantee (CHALLENGE).

What you already have

A real, working `IssueEditPanel` (title and status fields), a real `updateIssueAction`, a real persisted drag-and-drop Board, and the full real Sprint lifecycle UI. `PATCH /issues/:id` already accepts `priority` / `storyPoints` / `assigneeId` / `sprintId` in its body — confirm its real shape yourself (don't guess it). `DELETE /issues/:id` already exists and works.

What to build (required)

`IssueEditPanel` grows a real `"number"` row kind and a real `numeric?` option on its existing `"select"` kind, plus the real conversion `save()` needs to send a number (or `null`) instead of a raw string; three new `fields` entries (priority, assignee, story points) added at the Issue Detail call site; a real `deleteIssueAction` and a real, confirmed Delete button on Issue Detail; a real Backlog page, reachable from Project Detail, listing every issue with no sprint; Sprint choices and an assignment control for each Backlog issue; revalidation of affected Backlog, Sprint, and Board views; an authenticated Board read that renders only issues belonging to the active Sprint; a real story-points badge on the Board's own card.

CORE (required)

How to start

Grow `IssueEditPanel`'s own `EditableField` type and its input-rendering conditional in the SAME step — widening one without the other leaves TypeScript unable to narrow which row kind you are actually rendering. Only after that compiles, teach `save()` the real number-or-null conversion.

Hints

- **Hint 1 (conceptual):** every HTML form control hands back a plain string, even a real `<input type="number">`. Think about what the real backend's own validation does when it receives a numeric-looking string instead of an actual number.
- **Hint 2 (where to look):** the real assignee field is a `"select"`, not a new fourth kind — look at how `numeric?` is declared on the EXISTING `"select"` variant, not as a separate type.
- **Hint 3 (API or tool name):** `Number(value)`, and checking the emptied-input case (`value === ""`) before converting it.
- **Hint 4 (product flow):** load the project's Sprints, exclude completed Sprints from assignment choices, and reuse the existing issue-update path to persist `sprintId`.
- **Hint 5 (Board):** authenticate the Sprint read like the other protected reads. Find the active Sprint first, then derive the Board's issues from that Sprint's id.
- **Hint 6 (freshness and failure):** refresh every affected view after a successful change. A failed edit must restore the previous displayed value and show an inline error.

What to research

1. Why must the real `EditableField` type and the real input-rendering conditional that reads it change together, in one step, rather than across two?
2. What real, concrete value does `Number("")` actually produce, and why is that a genuine bug for a story-points field if left unhandled?

STRETCH (required)

How to start

Look at where `IssueCard` already renders on the Board, and where `IssueEditPanel`'s own story points field (just built in CORE) reads `issue.storyPoints`. The badge needs the identical real value, rendered in a second real place.

Hints

- **Hint 1 (conceptual):** `issue.storyPoints` is `number | null` — think about what the real card should show (or not show) when it is `null`.
- **Hint 2 (where to look):** `IssueCard` already receives the full real `issue` object as a prop; no new data fetch is needed anywhere.

CHALLENGE (required)

How to start

Re-read the real `SprintsService`'s own `start()` / `complete()` / `update()` methods (or the backend reference doc) and write down, in one sentence each, what real problem each of the three rules below actually prevents.

Hints

- **Hint 1 (conceptual):** think about what could go wrong for a real team's own reporting if two sprints were "active" in the same project at once.
- **Hint 2 (where to look):** compare the friendly service-level check with the PostgreSQL partial unique index. They solve different parts of the same invariant.

What to research

1. Why keep a service-level active-Sprint check **and** enforce the invariant with a PostgreSQL partial unique index? What does each layer provide?
2. Why can a completed sprint's own `name` / `goal` never be edited again?

Rules (required)

Allowed: everything already shown LIVE/TEAMWORK this session (the generic `fields`-array pattern, `@dnd-kit`, the full Sprint lifecycle actions, `useActionState`, `apiFetch`).

Not yet: no `@mention` picker, no Comments, no Activity feed, no Notifications UI, no Dashboard — all of that is Session 27.

Test it (required)

inside api/

BASH

```
PORT=3000 npm run start
```

inside web/ (second terminal)

BASH

```
npm run dev
```

in a third terminal

BASH

```
npx tsc --noEmit
npm run build
```

Then, in a real browser: open an issue, change its priority, assignee, and story points, and confirm each change persists after a reload; delete an issue and confirm it is actually gone; open `/backlog` and confirm it lists exactly the issues not currently in any sprint; assign one of those issues to a non-completed Sprint and confirm it leaves the Backlog; start that Sprint and confirm the Board displays only its issues; drag one of those issues to another status and reload to confirm persistence; confirm the Board's own card now shows a story-points badge. Also force a failed edit and confirm the previous value returns with an inline error.

Common confusions

- Adding a fourth `EditableField` kind for the assignee instead of reusing `"select"` with `numeric: true` — duplicates real conversion logic for no benefit.
- Converting with `parseInt` instead of `Number` — silently truncates a real non-integer string.
- Asking the real backend for a `sprintId` filter on the Backlog page — no such parameter exists; the filter is always client-side, over the already-fetched issue list.
- Assuming the Sprint lifecycle rules are UI conventions rather than real, independently server-enforced rules.
- Showing every project issue on the Board instead of deriving the list from the active Sprint.
- Offering a completed Sprint as an assignment target, or forgetting to refresh Backlog, Sprint, and Board views after assignment.

Done when (required)

- Priority, assignee, and story points are all really editable on Issue Detail, and persist.
- An issue can actually be deleted, with a real confirm step first.
- `/backlog` lists exactly the issues with no sprint, reachable from Project Detail.
- A Backlog issue can be assigned to a non-completed Sprint; affected views refresh, and the same update contract can move it back to Backlog with `sprintId = null`.
- The Board performs an authenticated Sprint read and shows only the active Sprint's issues; after a Sprint starts, drag-and-drop persists status changes for those issues.
- Failed edits roll back the displayed value and show an inline error.
- The Board's own card shows a real story-points badge.
- Your written CHALLENGE answer addresses all three Sprint rules (forward-only, one-active-per-project, frozen-on-completion), not just one of them.
- `npx tsc --noEmit` and `npm run build` both exit clean.

Working time (required)

About 99 minutes total (HW-01…26) — CORE (HW-01…24) is ~85 minutes; STRETCH (HW-25) is ~4 minutes; CHALLENGE (HW-26) is ~10 minutes.